Q1. Explain Conditional Statements with examples.

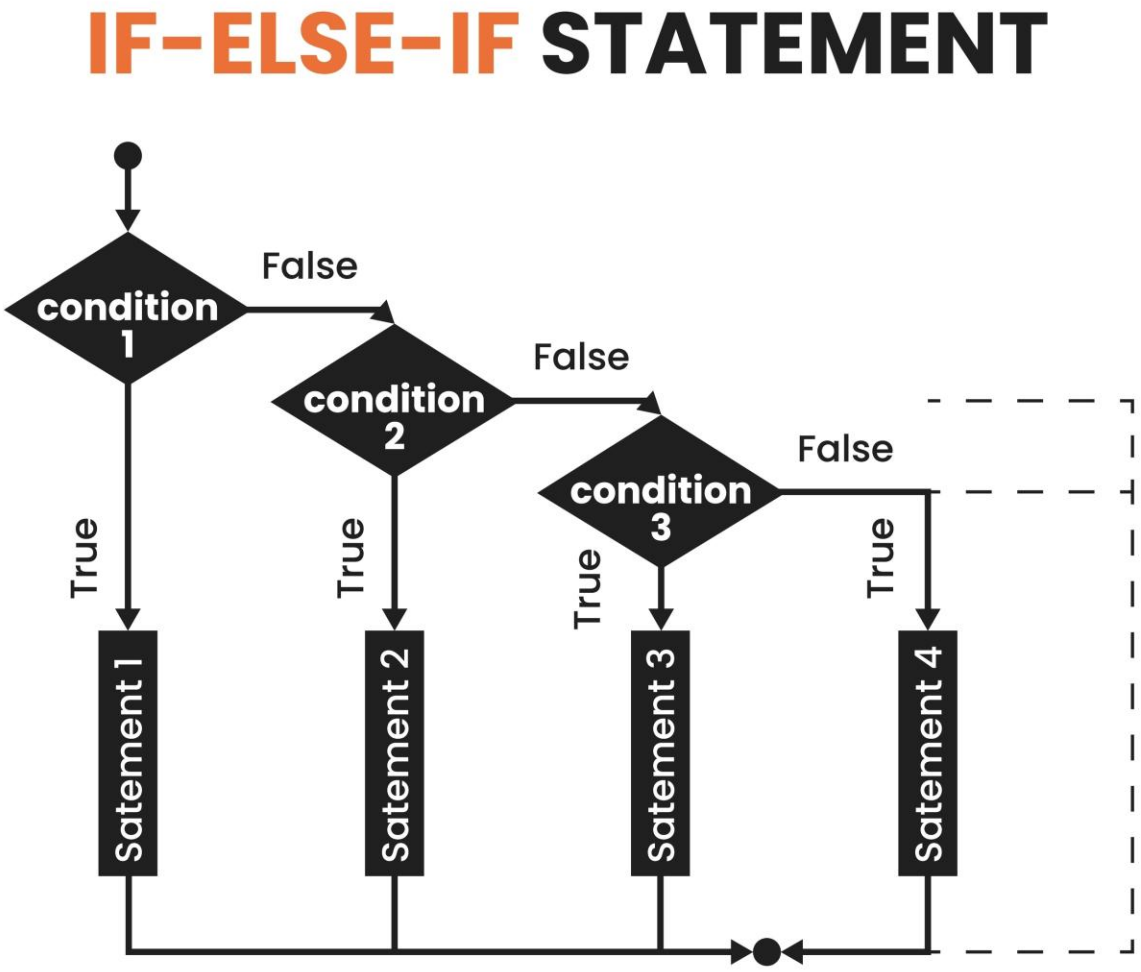
Answer:

1. Introduction

Conditional statements, also known as decision-making statements, allow a program to execute specific blocks of code based on whether a condition evaluates to **True** or **False**. In Python, these statements control the flow of execution, making the program "intelligent" enough to handle different inputs.

2. Types of Conditional Statements

- **If Statement:** The simplest form. It executes a block of code only if the condition is true.
- **If-Else Statement:** Provides an alternative. If the if condition is false, the else block executes.
- **If-Elif-Else Ladder:** Used when multiple conditions need to be checked sequentially.
- **Nested If:** An if statement inside another if statement for complex logic.



Shutterstock

3. Detailed Elaboration with Examples

- The if-elif-else Logic:

In a university grading system, we don't just have "pass" or "fail"; we have multiple ranges.

Python

score = 85

if score >= 90:

```
    print("Grade: A")
elif score >= 80:
    print("Grade: B") # This will execute
else:
    print("Grade: C")
```

- **Indentation:** Python uses whitespace (indentation) to define the scope of the block. Unlike C++ or Java which use curly braces { }, incorrect indentation in Python results in an IndentationError.

Q2. Explain Operators and types of operators with examples in detail.

Answer:

1. Definition

Operators are special symbols that carry out arithmetic or logical computations. The value that the operator operates on is called the **Operand**. For example, in $5 + 3$, the + is the operator and 5 and 3 are operands.

2. Classification of Operators

Type	Description	Examples
Arithmetic	Used for mathematical calculations.	+, -, *, /, % (Modulus), ** (Power), // (Floor Division)
Relational	Used to compare two values; returns True/False.	==, !=, >, <, >=, <=
Logical	Used to combine conditional statements.	and, or, not
Bitwise	Acts on bits and performs bit-by-bit operations.	& (AND), | (OR), ^ (XOR), ~ (NOT)
Assignment	Used to assign values to variables.	=, +=, -=, *=, /=
Special	Python-specific operators.	is (Identity), in (Membership)

3. Example Elaboration

- **Floor Division (//) vs Division (/):** $10 / 3$ results in 3.33, while $10 // 3$ results in 3 (integer portion only).
- **Membership Operators:** The in operator checks if a value exists within a sequence (like a list or string).

Python

fruits = ["apple", "banana"]

print("apple" in fruits) # Returns True

Q3. Explain Strings and All Their Functions in Detail in Python.

Answer:

1. Definition

A String in Python is a sequence of characters enclosed in single quotes (' '), double quotes (" "), or triple quotes (''' '''). Strings are **immutable**, meaning once created, their content cannot be changed.

2. String Slicing and Indexing

Python strings support both positive indexing (starting from 0) and negative indexing (starting from -1 for the last character).

- **Slicing:** string[start:stop:step]

3. Key String Functions (Methods)

- **lower() & upper():** Converts case.
- **strip():** Removes leading and trailing whitespaces.
- **replace(old, new):** Replaces a substring with another.
- **split(separator):** Breaks the string into a list based on the separator.
- **find(sub):** Returns the lowest index where the substring is found.

4. Example Usage

Python

```
text = " Python Programming "
print(text.strip().upper()) # Output: "PYTHON PROGRAMMING"
print(text.split())        # Output: ['Python', 'Programming']
```

Q4. Explain File, different types of files used, and modes of file in detail.

Answer:

1. Definition of a File

A file is a named location on a disk used to store data permanently. Python allows us to handle files through a built-in open() function.

2. Types of Files

- **Text Files (.txt, .py, .csv):** Store data as a sequence of characters (human-readable).
- **Binary Files (.jpg, .mp4, .exe):** Store data in the form of bytes (0s and 1s).

3. File Access Modes

Mode	Description
r	**Read Only:** Opens file for reading (default). Errors if file doesn't exist.
w	**Write Only:** Overwrites the file if it exists. Creates a new one if not.
a	**Append:** Adds data to the end of the file without overwriting.
r+	**Read & Write:** Opens for both reading and writing.
wb / rb	Opens binary files for writing or reading.

Q5. What is Type Conversion? Explain Implicit and Explicit with examples.

Answer:

1. Definition

Type conversion is the process of converting the value of one data type (integer, string, float, etc.) to another data type.

2. Implicit Type Conversion

In this, Python automatically converts one data type to another without user involvement. This usually happens to avoid data loss (converting a smaller type to a larger type).

- **Example:** Adding an integer to a float.

Python

```
a = 10    # int

b = 1.5   # float

c = a + b # Python converts 'a' to float automatically. c is 11.5
```

3. Explicit Type Conversion (Type Casting)

The user manually converts the data type using predefined functions like int(), float(), str().

- **Example:** Converting a string to an integer to perform math.

Python

```
val = "100"

num = int(val) + 50 # Explicitly cast string to int. Result: 150
```

Q6. Explain the difference between List and Tuple in Python.

Answer:

1. Comparison Table

Feature	**List**	**Tuple**
Mutability	**Mutable** (Can be changed).	**Immutable** (Cannot be changed).
Syntax	Defined with square brackets [ ].	Defined with parentheses ( ).
Memory	Consumes more memory.	Consumes less memory.
Performance	Slower.	Faster (Used for fixed data).
Methods	Many methods (append, remove, pop).	Few methods (count, index).

2. Block Diagram Representation

Plaintext

List (Mutable) Tuple (Constant)

[1, 2, 3] (1, 2, 3)

| |

[1, 9, 3] (Allowed) (1, 9, 3) (Error!)

3. When to use which?

- Use a **List** when you have a collection of data that needs to be updated or changed frequently (e.g., a shopping cart).
- Use a **Tuple** for data that should remain constant throughout the program execution (e.g., coordinates of a location).